Palm vein module SDK usage documentation

Palm vein module SDK usage documentation

1. Import SDK
   - 1. Copy the aar file to the libs folder and put the .bin resource file into the assets directory;
   - 2. Add to dependencies in build.gradle
2. SDK Authorization
   - 1. Authorization through modules
   - 2. Authorization via the network
   - 3. Authorization via security chip
3. Find USB devices
4. Initialize the device
   - 1. Initialize in root mode
   - 2. Non- root initialization
5. Initialize SDK
6. Palm vein registration
   - 1. Open the recording mode
   - 2. Set up image capture and register callback
   - 3. End of registration
7. Add or delete palm veins
   - 1. Initialize the base database, where group is the group name.
   - 2. Add features to the database. Note that this method does not store them permanently, but only stores them in memory for comparison.
   - 3. Remove the palmar vein from the base
8. Palm vein recognition
   - 1. Set up image capture and register callbacks
   - 2.Comparison
9. Parameter settings
10. Algorithm version number
    - 1. Get the algorithm version number
    - 2. Determine whether the algorithm version is compatible
11. Non-module usage
12. Error Code

Date	Revisions	Modifications	Author
2022.12.01	V1.0	SDK Calling Method	Lin Longqing
2022.12.05	V1.1	Add SDK authorization	Lin Longqing
2023.02.14	V1.2	Add USB authorization and USB management	Lin Longqing
2023.12.22	V2.0	Updated to the second generation algorithm	Lin Longqing
2024.04.19	V2.1	Optimize image memory and develop algorithm version number interface	Lin Longqing

Date	Revisions	Modifications	Author
2024.05.16	V2.2	Add external interface description	Lin Longqing
2024.05.16	V2.3	Add module authorization instructions	Lin Longqing
2024.09.03	V2.4	Add non-module usage instructions	Lin Longqing
2025.02.26	V2.5	Added security chip authorization	Lin Longqing

1. Import SDK

1. Copy the aar file to the libs folder and put the .bin resource file into the assets directory;

2. Add the following to the build.gradle dependencies

```
implementation(fileTree("libs"));
```

2.SDK Authorization

PV920 can be authorized by module, PV910 and PV720 can be authorized by network

1. Authorization through modules

No additional action is required, but the device must be initialized before initializing the SDK, and the device needs to be initialized via JXPalmApi.ANDROID_915_OTG_DEVICE, etc.

```
int retVal = JXPalmTool.getPalmInstance().initializeDevice(
    this, device, JXPalmApi.ANDROID_ 915_OTG_DEVICE);
int ret = JXPalmTool.getPalmInstance().initializeSDK(...);
```

2. Authorization via the network

You need to obtain authorization before use, including APP_ID and SIGN_KEY . When authorizing the device, you need network permission and storage read and write permission.
Add the following statement to AndroidMainifest.xml :

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.READ_E XTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE "/>
```

The authorization code is as follows

```
int result = JXLicense.init(getApplicationContext());
if(result == 0){
    initPalmSDK();
}else{
```

```
    JXAuthen.getKey(APP_ID, SIGN_KEY, new JXAuthen.AuthenListener() {
        @Override
        public void complete(int code, String message, String data ) {
            int result = JXLicense.setAuthFile(getApplicationContext(), data);
            initPalmSDK();
        }
    });
}
```

3. Authorization via security chip

```
byte[] code  = new byte[112];
byte[] license  = new byte[224];
JXLicense.getICAuthenCode(code, license);
String codeStr = 加密芯片解密
String modelStr = 加密芯片解密
int ret = JXLicense.setICAuthenLicense(codeStr, modelStr);
```

3. Find USB devices

1. Initialize the USB device manager

```
JXPalmUSBManager usbManager =
    new JXPalmUSBManager(this.getApplicationContext(),usbListener);
usbManager.registerUSBPermissionReceiver();
usbManager.initUSBPermission();
```

2. USB monitoring callback: When the palm vein device is found, it will be called back in onCheckPermission . When the device is disconnected, it will be called back in onUSBRemoved. Callback in onUSBArrived after the device is inserted

```
private JXPalmUSBManagerListener usbListener =
    new JXPalmUSBManagerListener(){
        public void onCheckPermission (int result, UsbDevicedevice){}
        public void onUSBArrived(UsbDevice device) {}
        public void onUSBRemoved(UsbDevice device) {}
    }
```

4. Initialize the device

1. Initialize in root mode

Obtain USB permissions. Different Android motherboards may have different ways of obtaining permissions. The following is only for reference:

```
static public void getPermission() {
    String[] commands = new String[] {"chmod -R 777 /dev/bus/usb" };
    Process = null;
    DataOutputStream dataOutputStream = null;
    try {
        process = Runtime.getRuntime().exec("su");
        dataOutputStream = new DataOutputStream(process.getOutputStream());

        int length = commands.length;
        for (int i = 0; i < length; i++) {
            dataOutputStream.writeBytes(commands[i] + "\n");
        }
        dataOutputStream.writeBytes("exit\n");
        dataOutputStream.flush();
        process.waitFor();
        Log.i("USBPermissionManager", "getPermission success");
    } catch (Exception e) {
        FileLogUtil.e("USBPermissionManager", "Exception " + e);
    } finally {
        try {
            if (dataOutputStream != null) {
                dataOutputStream.close();
            }
            process.destroy();
        } catch (Exception e) {

        }
    }
}
```

Initialize the palm vein device. To enable the device, you need to obtain the USB permission:

```
int retVal = JXPalmTool.getPalmInstance().initializeDevice(
    this, device, NULL);
```

2. Non- root initialization

The last parameter cannot be empty, and the options are

ANDROID_910_DEVICE , ANDROID_915_DEVICE , ANDROID_915_OTG_DEV ICE

```
int retVal = JXPalmTool.getPalmInstance().initializeDevice(
    this, device, JXPalmApi.ANDROID_ 915_OTG_DEVICE);
```

5. Initialize SDK

Initialization needs to be performed after authorization is completed. The initialization process is a time-consuming operation. It is recommended to put it into an asynchronous thread for initialization. The path needs to refer to the actual Actual file name

```
int ret =JXPalmTool.getPalmInstance().initializeSDK(this,
        "palm/palm_detection_0904_en.bin",
        "palm/palm_filter_en.bin ",
        "palm/palm_landmark_0904_en.bin",
        "palm/palm_ir_liveness_20231215_en.bin",
        "palm/palm_vein_feature_1121_2_en.bin");
```

6. Palm vein registration

1. Open the recording mode

```
JXPalmTool.getPalmInstance().startEnroll();
```

2. Set up image capture and register callback

Set the callback function:

```
JXPalmTool.getPalmInstance( ).setEnrollListener(listener);
```

Start capturing images . If you want to adjust the distance detection rate or image capture rate, you need to add blocking code in the callback function , for example: Thread.Sleep()

```
private JXPalmCaptureListener mCaptureListener = new JXPalmCaptureListener() {
    @Override
    public void onDistance(int i) {
    }

    @Override
    public void onCapture(int code, JXimax jxImage) {
    }
};

JXPalmTool.getPalmInstance().startCapture(mCaptureListener);
```

Parameter name	Parameter Type	More Information
code	int	Error code , 0 success, other failure
image	JXImage	480 width , 640 height grayscale image

Callback function:

```
final JXPalmEnrollListener listener = new JXPalmEnrollListener() {
    @Override
    public void onEnroll(int code, int index, byte[] feature, float
    motion, JXImage image) {}

    @Override
    public void onDetected(int code, int[] rect, JXImage
    jxImage) {}

    @Override
    public void onLiveness(int code , float score) {}
};
JXPalmTool.getPalmInstance().startEnroll();
```

onEnroll

Parameter name	Parameter Type	More Information
code	int	Error code, 0 success, 359 incomplete palm, 354 , 358 palm distance is too far, 352 palm brightness is too dim , 353 The palm brightness is too bright, 355 the palm shape does not meet expectations (e.g., tilted, joined palms, etc.)
index	int	Input subscript , representing the number of inputs
feature	int	The length of the vein feature is 2048 bytes
image	JXImage	480 width , 640 height grayscale image

onDetected

Parameter name	Parameter Type	More Information
code	int	Error code, 0 for success, other for failure
rect	int[]	Palm position [x1, y1, x2, y2, x3, y3 , x4, y4]
image	JXImage	480 width , 640 height grayscale image

onLiveness

Parameter name	Parameter Type	More Information
code	int	Error code , 0 for success, other for failure
score	float	Liveness score greater than 0.8 is alive

3. End of registration

```
JXPalmTool.getPalmInstance().cancelEnroll();
```

7. Add or delete palm veins

1. Initialize the database, where ‘group’ is the group name.

```
JXPalmTool.getPalmInstance().initGroup(String group);
```

2. Add features to the database. Note that this method does not store them permanently, but only in memory for comparison purposes.

```
JXPalmTool.getPalmInstance().addPalm(String group, String uid,
    int index, byte[] feature);
```

Parameter name	Parameter Type	More Information
group	String	Compare the group name of the database
uid	String	Palm vein ID or user ID
index	int	Palm type, 0 for left hand and 1 for right hand.But not strictly limited.
feature	byte[]	Palm vein characteristics

3. Remove the palm vein from the database

```
JXPalmTool.getPalmInstance().delPalm(C String group, String uid,
    int index);
```

8. Palm vein recognition

1. Set up image capture and register callback

Set the callback function:

```
JXPalmTool.getPalmInstance( ).setMatchListener(listener);
```

Start capturing images

```
JXPalmTool.getPalmInstance().startCapture((code, image)->{
    if (0 == code) {
        Bitmap bitmapPalm = image.toBitmap();
    // Display the image
   }
});
```

Note: If jxImage is not in use or has been used up, please call jxImage.recycle() to improve memory usage efficiency and avoid frequent triggering of GC .

Callback function:

```
final JXPalmMatchListener mPalmMatchListener = new JXPalmMatchListener() {
        @Override
        public void onMatch(int code, byte[] feature, JXImage jxI mage) {}

        @Override
        public void onDetected(int code , int[] rect, JXImage jxImage) {}

        @Override
        public void onLiveness(int result, float score) {}
    };
```

onMatch

Parameter Name	Parameter Type	Details
code	int	Error code, 0 for success, others for failure
feature	int	The length of the vein feature is 2048 bytes
image	JXImage	width 480 , height 640 grayscale image

Note: If jxImage is not in use or has been used up, please call jxImage.recycle() to improve memory usage efficiency and avoid frequent triggering of GC .

onDetected

Parameter name	Parameter Type	More Information
code	int	Error code, 0 for success, others for failure
rect	int[]	Palm position [x1, y1, x2, y2, x3, y3 , x4, y4]
image	JXImage	Width 480 , height 640 grayscale image

Note: If jxImage is not in use or has been used up, please call jxImage.recycle() to improve memory usage efficiency and avoid frequent triggering of GC .

onLiveness onLiveness

Parameter name	Parameter Type	More Information
code	int	Error code , 0 for success, other for failure
score	float	Liveness score , greater than 0.8 is alive

2. Comparison

```
int retVal = JXPalmTool.getPalmInstance().palmMatch(String group, byte[] feature,
String[] uuids);
```

Parameter name	Parameter Type	More Information
group	String	Compare the group name of the database
feature	byte[]	Palm vein characteristics
uuids	String[]	Palm vein ID or user ID . The current version returns top1 , so the length is 1

9. Parameter settings

JXPalmConfig

Parameter name	Parameter Type	More Information
IrLivenessEnable	boolean	IR liveness enabled?
Rotation	int	Rotation Angle
HorizonFlip	boolean	Horizontal mirror
VerticalFlip	boolean	Mirror vertically
MinEnrollQuality	int	The minimum quality of the input allowed
MinEnrollBrightness	int	The minimum recording brightness allowed
MaxEnrollBrightness	int	The maximum recording brightness allowed
MaxEnrollRoll	int	Maximum rotation angle allowed to be entered
MaxEnrollYaw	int	Maximum horizontal rotation angle allowed to be entered
MaxEnrollPitch	int	Maximum horizontal rotation angle allowed to be entered
MinEnrollSize	int	Minimum size allowed for entry
MinEnrollOccupyRatio	float	Minimum percentage allowed to be entered
MinPointResponse	float	The minimum key point corresponding to the input
MinVerifyQuality	int	Minimum image quality
MinVerifyBrightness	int	Minimum brightness

Parameter name	Parameter Type	More Information
MaxVerifyBrightness	int	Maximum brightness
MaxVerifyRoll	int	Maximum rotation angle
MaxVerifyYaw	int	Maximum horizontal rotation angle
MaxVerifyPitch	int	Maximum vertical rotation angle
MinVerifySize	int	Minimum verification size
MinVerifyOccupyRatio	float	Minimum verification ratio
LivenessThreshold	float	Liveness threshold

10. Algorithm version number

1. Get the algorithm version number

```
JXPalmApi.getAlgoVersion()
```

Return value example: 2.0.0 major.minor.patch

2. Determine whether the algorithm version is compatible

```
JXPalmApi.isAlgoVersionCompatible("2.1.0")
```

If true is returned , it means it is compatible, otherwise it is not compatible. In principle, the major version remains unchanged and the extracted features are compatible.

11. Non-module usage

Non-modules do not need to initialize the device, that is, do not call initializeDevice , but start the worker thread

```
JXPalmTool.getPalmInstance().startWork()
```

If you need to stop the working thread , call the following function

```
JXPalmTool.getPalmInstance().stopWork()
```

After starting the worker thread, you need to input the image from the outside

```
JXPalmTool.getPalmInstance().addDetectImage(JXImage image);
```

JXImage can be converted from

```
JXPalmTool.getPalmInstance().yuvToJXImage(byte[ ] data, int width, int height);
```

12. Error Code

Error Code	More Information
101	Parameter length error
102	Wrong resource path
103	SDK not initialized
104	Parameter error
105	Parameter not supported
106	Initialization Error
107	Handle Error
109	Memory Error
201	The number of image channels is not supported
301	Palm not detected
302	Palm filter failed
303	Live body not passed
351	Image quality failed
352	Image brightness is too low
353	Image brightness is too high
354	The palm ratio is too small
355	Palm gesture detection failed
358	The palm area is too small
359	Incomplete palm
20001	Authorization Error